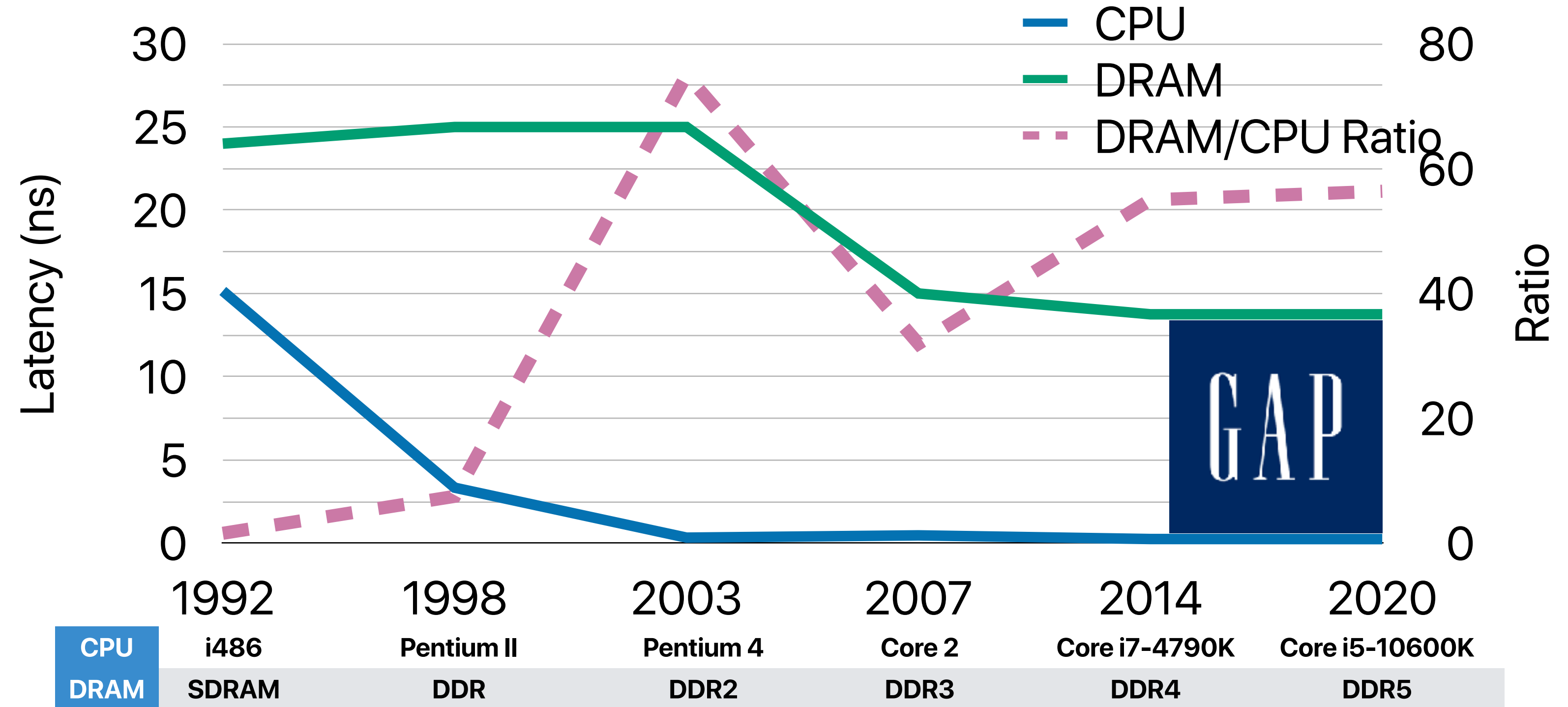


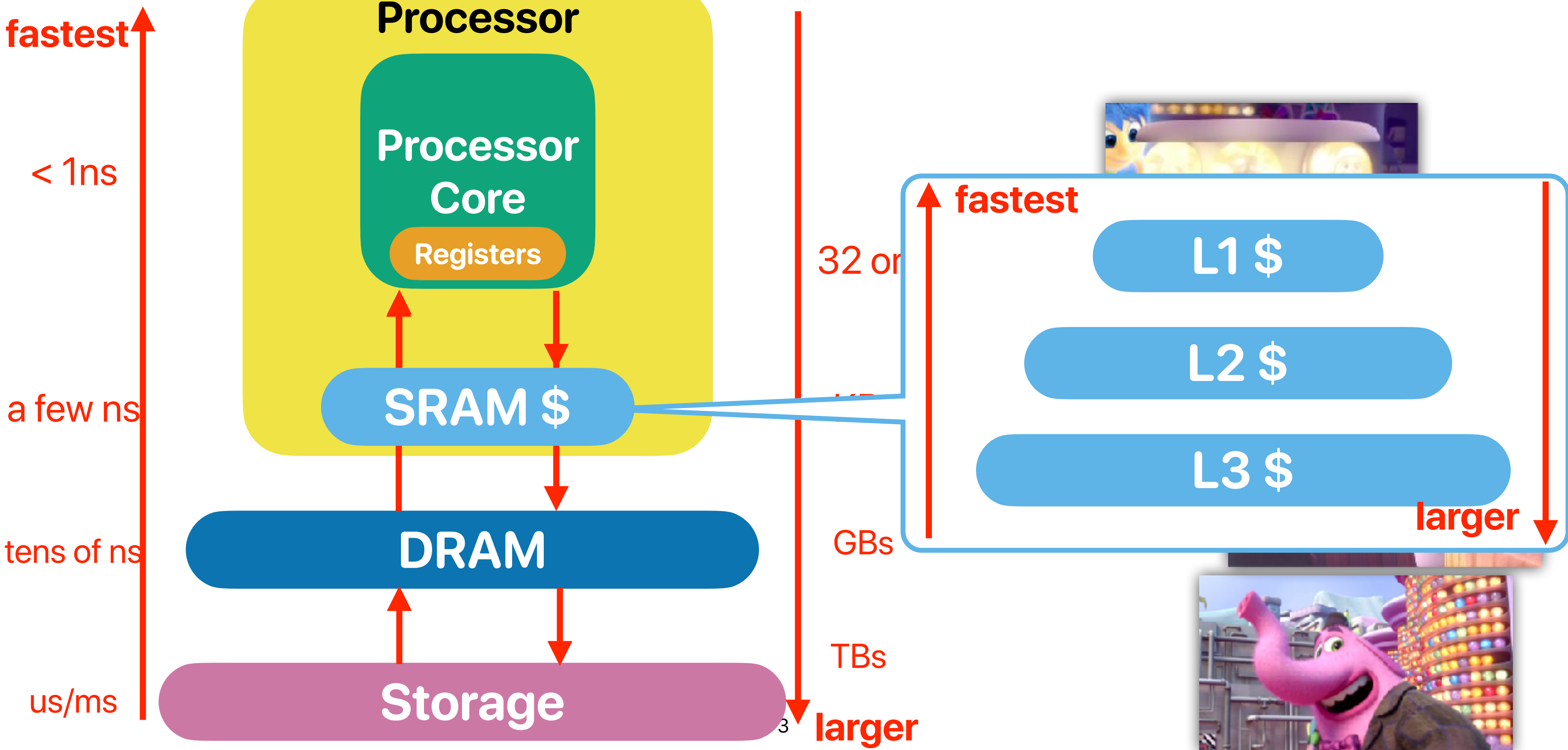
Memory Hierarchy (3): Cache Misses and How to Address Them

Hung-Wei Tseng

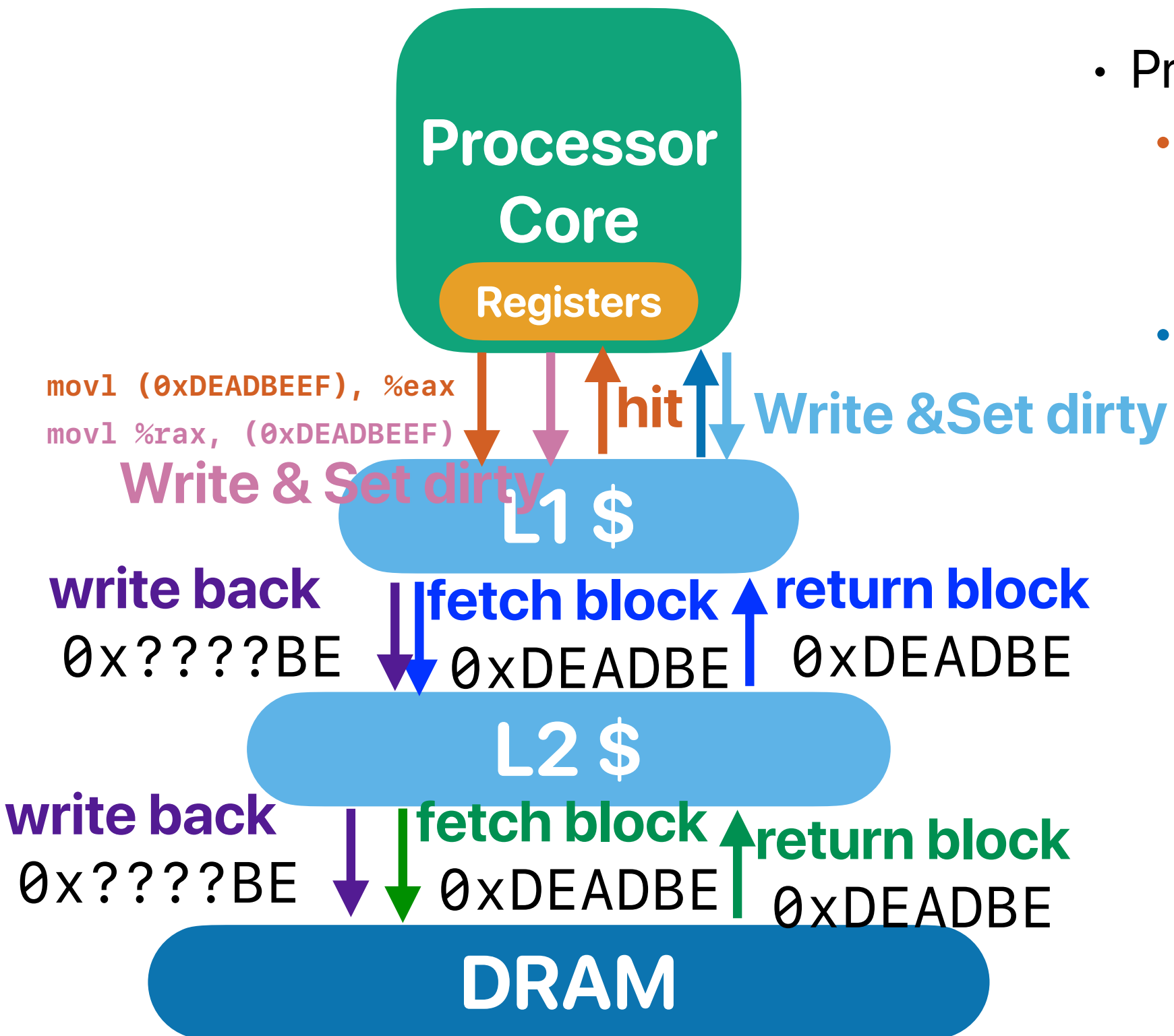
Recap: the "latency" gap between CPU and DRAM



Recap: Memory Hierarchy



Recap: Processor/cache interaction



- Processor sends memory access request to L1-\$
 - **if hit & it's a read**
 - **Read: return data**
 - **Write: Update "ONLY" in L1 and set DIRTY**
 - **if miss**
 - If there an empty block — place the data there
 - If NOT (most frequent case) — select a **victim block**
 - Least Recently Used (LRU) policy
 - If the victim block is "dirty" & "valid"
 - **Write back** the block to lower-level memory hierarchy
 - If write-back or fetching causes any miss, repeat the same process
 - Fetch the requesting block from lower-level memory hierarchy and place in the cache
 - **Present the write "ONLY" in L1 and set DIRTY**

Simulate a direct-mapped cache

tag index

	V	D	Tag	Data
0	0	0		
1	0	0		
2	0	0		
3	1	0	0x558FE0A1DC	b[0], b[1]
4	1	0	0x558FE0A1DC	b[2], b[3]
5	0	0		
6	0	0		
7	0	0		
8	0	0		
9	0	0		
10	0	0		
11	0	0		
12	0	0		
13	0	0		
14	0	0		
15	0	0		

This cache doesn't work!!!
— collisions!

Address (Hex)		
&a[0][0]	0x558FE0A1D330	miss
&b[0]	0x558FE0A1DC30	miss
&a[0][1]	0x558FE0A1D338	miss
&b[1]	0x558FE0A1DC38	miss
&a[0][2]	0x558FE0A1D340	miss
&b[2]	0x558FE0A1DC40	miss
&a[0][3]	0x558FE0A1D348	miss
&b[3]	0x558FE0A1DC48	miss
&a[0][4]	0x558FE0A1D350	miss
&b[4]	0x558FE0A1DC50	miss
&a[0][5]	0x558FE0A1D358	miss
&b[5]	0x558FE0A1DC58	miss
&a[0][6]	0x558FE0A1D360	miss
&b[6]	0x558FE0A1DC60	miss
&a[0][7]	0x558FE0A1D368	miss
&b[7]	0x558FE0A1DC68	miss
&a[0][8]	0x558FE0A1D370	miss
&b[8]	0x558FE0A1DC70	miss
&a[0][9]	0x558FE0A1D378	
&b[9]	0x558FE0A1DC78	

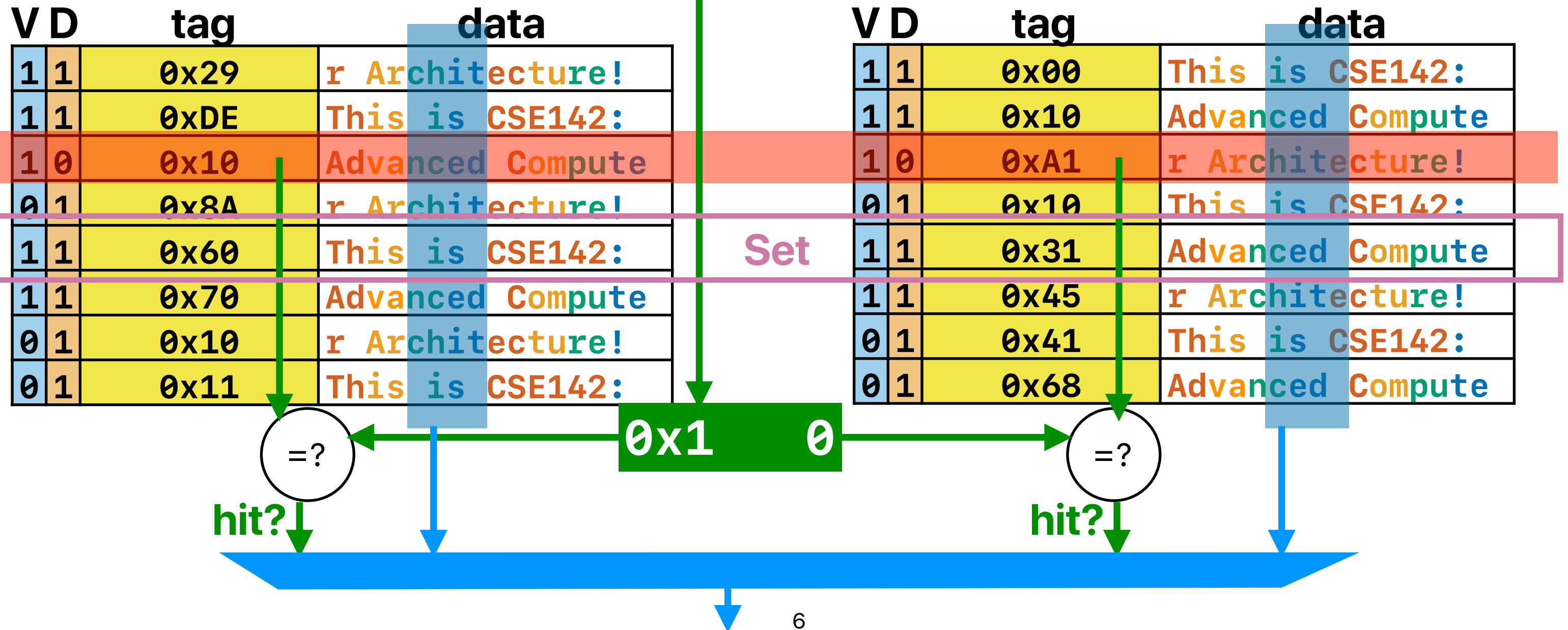
Recap: Way-associative cache

memory address: $0x0$ 8 2 4

set block

tag index offset

memory address: $0b00001000000100100$



Simulate a 2-way cache

V	D	Tag	Data	V	D	Tag	Data
0	0			0	0		
0	0			0	0		
0	0			0	0		
1	0	0xAB1FC143A6	a[0][0], a[0][1]	1	0	0xAB1FC143B8	b[0], b[1]
1	0	0xAB1FC143A6	a[0][2], a[0][3]	1	0	0xAB1FC143B8	b[2], b[3]
0	0			0	0		
0	0			0	0		
0	0			0	0		

	Address (Hex)	Tag	Index	
&a[0][0]	0x558FE0A1D330	0xAB1FC143A6	0x3	miss
&b[0]	0x558FE0A1DC30	0xAB1FC143B8	0x3	miss
&a[0][1]	0x558FE0A1D338	0xAB1FC143A6	0x3	hit
&b[1]	0x558FE0A1DC38	0xAB1FC143B8	0x3	hit
&a[0][2]	0x558FE0A1D340	0xAB1FC143A6	0x4	miss
&b[2]	0x558FE0A1DC40	0xAB1FC143B8	0x4	miss
&a[0][3]	0x558FE0A1D348	0xAB1FC143A6	0x4	hit
&b[3]	0x558FE0A1DC48	0xAB1FC143B8	0x4	hit
&a[0][4]	0x558FE0A1D350	0xAB1FC143A6	0x5	miss
&b[4]	0x558FE0A1DC50	0xAB1FC143B8	0x5	miss
&a[0][5]	0x558FE0A1D358	0xAB1FC143A6	0x5	hit
&b[5]	0x558FE0A1DC58	0xAB1FC143B8	0x5	hit
&a[0][6]	0x558FE0A1D360	0xAB1FC143A6	0x6	miss
&b[6]	0x558FE0A1DC60	0xAB1FC143B8	0x6	miss
&a[0][7]	0x558FE0A1D368	0xAB1FC143A6	0x6	hit
&b[7]	0x558FE0A1DC68	0xAB1FC143B8	0x6	hit
&a[0][8]	0x558FE0A1D370	0xAB1FC143A6	0x7	miss
&b[8]	0x558FE0A1DC70	0xAB1FC143B8	0x7	miss
&a[0][9]	0x558FE0A1D378	0xAB1FC143A6	0x7	hit
&b[9]	0x558FE0A1DC78	0xAB1FC143B8	0x7	hit

Recap: $C = ABS$

- **C**: Capacity in data arrays
- **A**: Way-Associativity — how many blocks within a set
 - N-way: N blocks in a set, $A = N$
 - 1 for direct-mapped cache
- **B**: Block Size (Cacheline)
 - How many bytes in a block
- **S**: Number of Sets:
 - A set contains blocks sharing the same index
 - 1 for fully associate cache
- number of bits in **block** offset — $\lg(B)$
- number of bits in **set** index: $\lg(S)$
- tag bits: $\text{address_length} - \lg(S) - \lg(B)$
 - address_length is N bits for N-bit machines (e.g., 64-bit for 64-bit machines)
- $(\text{address} / \text{block_size}) \% S = \text{set index}$



Recap: designing caches

- Basic cache structures —
 - Caching in granularity of a block to capture spatial locality
 - Caching multiple blocks to keep frequently used data — temporal locality
 - Tags to distinguish cached blocks
- Hierarchical caching — data must be presented on the top level (L1) before the processor can use
- Optimizing cache structures
 - Hash block into "sets" to reduce the search time
 - Set-associativity to reduce the "collision" problem
- $C = A B S$
 - C: capacity
 - A: Associativity
 - S: Number of sets
 - $\lg(S)$: Number of bits in set index
 - $\lg(B)$: Number of bits in block offset

During examines/interviews, why do you (occasionally) don't know the answer?

Outline

- Estimating how cache friendly is our code
- The sources of cache misses
- Writing cache-friendly code

Estimating code performance on caches (cont.)

NVIDIA Tegra Orin

100% miss rate!

- Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
64KB = 4 * 64 * S
S = 256
offset = lg(64) = 6 bits
index = lg(256) = 8 bits
tag = the rest bits

	Address (Hex)	Address in binary			Tag	Index	Hit? Miss?	Replace?
a[0]	0x10000	0b000100	00000000	0000000	0x4	0x0	Miss	
b[0]	0x20000	0b001000	00000000	0000000	0x8	0x0	Miss	
c[0]	0x30000	0b001100	00000000	0000000	0xC	0x0	Miss	
d[0]	0x40000	0b010000	00000000	0000000	0x10	0x0	Miss	
e[0]	0x50000	0b010100	00000000	0000000	0x14	0x0	Miss	a[0-7]
a[1]	0x10008	0b000100	00000000	001000	0x4	0x0	Miss	b[0-7]
b[1]	0x20008	0b001000	00000000	001000	0x8	0x0	Miss	c[0-7]
c[1]	0x30008	0b001100	00000000	001000	0xC	0x0	Miss	d[0-7]
d[1]	0x40008	0b010000	00000000	001000	0x10	0x0	Miss	e[0-7]
e[1]	0x50008	0b010100	00000000	001000	0x14	0x0	Miss	a[0-7]
⋮	⋮	⋮			⋮	⋮	⋮	⋮

NVIDIA Tegra Orin

- D-L1 Cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

- A. 12.5%
- B. 56.25%
- C. 66.67%
- D. 68.75%
- E. 100%



intel Core i7

- D-L1 Cache configuration of intel Core i7
 - Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

- A. 12.5%
- B. 56.25%
- C. 66.67%
- D. 68.75%
- E. 100%



intel Core i7

- Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
48KB = 12 * 64 * S
S = 64
offset = lg(64) = 6 bits
index = lg(12) = 3.58 bits
tag = the rest bits

	Address (Hex)	Address in binary	Tag	Index	Hit? Miss?	Replace?
a[0]	0x10000	0b0001000000000000000000	0x10	0x0	Miss	
b[0]	0x20000	0b0010000000000000000000	0x20	0x0	Miss	
c[0]	0x30000	0b0011000000000000000000	0x30	0x0	Miss	
d[0]	0x40000	0b0100000000000000000000	0x40	0x0	Miss	
e[0]	0x50000	0b0101000000000000000000	0x50	0x0	Miss	
a[1]	0x10008	0b0001000000000000001000	0x10	0x0	Hit	
b[1]	0x20008	0b0010000000000000001000	0x20	0x0	Hit	
c[1]	0x30008	0b0011000000000000001000	0x30	0x0	Hit	
d[1]	0x40008	0b0100000000000000001000	0x40	0x0	Hit	
e[1]	0x50008	0b0101000000000000001000	0x50	0x0	Hit	
⋮	⋮	⋮	⋮	⋮	⋮	⋮

intel Core i7 (cont.)

- Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
48KB = 12 * 64 * S
S = 64
offset = lg(64) = 6 bits
index = lg(64) = 6 bits
tag = the rest bits

	Address (Hex)	Address in binary			Tag	Index	Hit? Miss?	Replace?
a[7]	0x10038	0b00010000	000000	111000	0x10	0x0	Hit	
b[7]	0x20038	0b00100000	000000	111000	0x20	0x0	Hit	
c[7]	0x30038	0b00110000	000000	111000	0x30	0x0	Hit	
d[7]	0x40038	0b01000000	000000	111000	0x40	0x0	Hit	
e[7]	0x50038	0b01010000	000000	111000	0x50	0x0	Hit	
a[8]	0x10040	0b00010000	000001	000000	0x10	0x1	Miss	
b[8]	0x20040	0b00100000	000001	000000	0x20	0x1	Miss	
c[8]	0x30040	0b00110000	000001	000000	0x30	0x1	Miss	
d[8]	0x40040	0b01000000	000001	000000	0x40	0x1	Miss	
e[8]	0x50040	0b01010000	000001	000000	0x50	0x1	Miss	
a[9]	0x10048	0b00010000	000001	001000	0x10	0x1	Hit	
b[9]	0x20048	0b00100000	000001	001000	0x20	0x1	Hit	
c[9]	0x30048	0b00110000	000001	001000	0x30	0x1	Hit	
d[9]	0x40048	0b01000000	000001	001000	0x40	0x1	Hit	

$$\frac{5 \times \frac{512}{8}}{5 \times 512} = \frac{1}{8} = 12.5 \%$$

Miss when the array index is a multiply of 8!

Taxonomy/reasons of cache misses

3Cs of misses

- Compulsory miss
 - Cold start miss. First-time access to a block
- Capacity miss
 - The working set size of an application is bigger than cache size
- Conflict miss
 - Required data replaced by block(s) mapping to the same set
 - Similar collision in hash



NVIDIA Tegra Orin

- D-L1 Cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the major type of cache misses when running the above code snippet?

- A. Compulsory misses
- B. Conflict misses
- C. Capacity misses
- D. Capacity and conflict misses
- E. They are all equally critical



NVIDIA Tegra Orin

100% miss rate!

- Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
64KB = 4 * 64 * S
S = 256
offset = lg(64) = 6 bits
index = lg(256) = 8 bits
tag = the rest bits

	Address (Hex)	Address in binary	Tag	Index	Hit? Miss?	Replace?
a[0]	0x10000	0b0001000000000000000000	0x8	0x0	Compulsory Miss	
b[0]	0x20000	0b0010000000000000000000	0x10	0x0	Compulsory Miss	
c[0]	0x30000	0b0011000000000000000000	0x18	0x0	Compulsory Miss	
d[0]	0x40000	0b0100000000000000000000	0x20	0x0	Compulsory Miss	
e[0]	0x50000	0b0101000000000000000000	0x28	0x0	Compulsory Miss	a[0-7]
a[1]	0x10008	0b0001000000000000001000	0x8	0x0	Conflict Miss	b[0-7]
b[1]	0x20008	0b0010000000000000001000	0x10	0x0	Conflict Miss	c[0-7]
c[1]	0x30008	0b0011000000000000001000	0x18	0x0	Conflict Miss	d[0-7]
d[1]	0x40008	0b0100000000000000001000	0x20	0x0	Conflict Miss	e[0-7]
e[1]	0x50008	0b0101000000000000001000	0x28	0x0	Conflict Miss	a[0-7]
⋮	⋮	⋮	⋮	⋮	⋮	⋮

NVIDIA Tegra Orin (cont.)

- Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
 64KB = 4 * 64 * S
 S = 256
 offset = lg(64) = 6 bits
 index = lg(256) = 8 bits
 tag = the rest bits

	Address (Hex)	Address in binary	Tag	Index	Hit? Miss?	Replace?
a[7]	0x10038	0 b0001000 0000000 111000	0x8	0x0	Conflict Miss	
b[7]	0x20038	0 b0010000 0000000 111000	0x10	0x0	Conflict Miss	
c[7]	0x30038	0 b0011000 0000000 111000	0x18	0x0	Conflict Miss	
d[7]	0x40038	0 b0100000 0000000 111000	0x20	0x0	Conflict Miss	
e[7]	0x50038	0 b0101000 0000000 111000	0x28	0x0	Conflict Miss	a[0-7]
a[8]	0x10040	0 b0001000 000000 1000000	0x8	0x1	Compulsory Miss	
b[8]	0x20040	0b00100000000000 1000000	0x10	0x1	Compulsory Miss	
c[8]	0x30040	0b00110000000000 1000000	0x18	0x1	Compulsory Miss	
d[8]	0x40040	0b01000000000000 1000000	0x20	0x1	Compulsory Miss	
e[8]	0x50040	0b01010000000000 1000000	0x28	0x1	Compulsory Miss	a[8-15]

100% miss rate!

NVIDIA Tegra Orin

- D-L1 Cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the major type of cache misses when running the above code snippet?

- A. Compulsory misses
- B. Conflict misses
- C. Capacity misses
- D. Capacity and conflict misses
- E. They are all equally critical



intel Core i7

- D-L1 Cache configuration of intel Core i7
 - Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the major type of cache misses when running the above code snippet?

- A. Compulsory misses
- B. Conflict misses
- C. Capacity misses
- D. Capacity and conflict misses
- E. They are all equally critical



intel Core i7

- Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
48KB = 12 * 64 * S
S = 64
offset = lg(64) = 6 bits
index = lg(64) = 6 bits
tag = the rest bits

	Address (Hex)	Address in binary	Tag	Index	Hit? Miss?	Replace?
a[0]	0x10000	0b0001000000000000000000	0x10	0x0	Compulsory Miss	
b[0]	0x20000	0b0010000000000000000000	0x20	0x0	Compulsory Miss	
c[0]	0x30000	0b0011000000000000000000	0x30	0x0	Compulsory Miss	
d[0]	0x40000	0b0100000000000000000000	0x40	0x0	Compulsory Miss	
e[0]	0x50000	0b0101000000000000000000	0x50	0x0	Compulsory Miss	
a[1]	0x10008	0b0001000000000000001000	0x10	0x0	Hit	
b[1]	0x20008	0b0010000000000000001000	0x20	0x0	Hit	
c[1]	0x30008	0b0011000000000000001000	0x30	0x0	Hit	
d[1]	0x40008	0b0100000000000000001000	0x40	0x0	Hit	
e[1]	0x50008	0b0101000000000000001000	0x50	0x0	Hit	
⋮	⋮	⋮	⋮	⋮	⋮	⋮

intel Core i7 (cont.)

- Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
```

C = ABS
48KB = 12 * 64 * S
S = 64
offset = lg(64) = 6 bits
index = lg(64) = 6 bits
tag = the rest bits

	Address (Hex)	Address in binary	Tag	Index	Hit? Miss?	Replace?
a[7]	0x10038	0b00010000000000111000	0x10	0x0	Hit	
b[7]	0x20038	0b00100000000000111000	0x20	0x0	Hit	
c[7]	0x30038	0b00110000000000111000	0x30	0x0	Hit	
d[7]	0x40038	0b01000000000000111000	0x40	0x0	Hit	
e[7]	0x50038	0b01010000000000111000	0x50	0x0	Hit	
a[8]	0x10040	0b00010000000001000000	0x10	0x1	Compulsory Miss	
b[8]	0x20040	0b00100000000001000000	0x20	0x1	Compulsory Miss	
c[8]	0x30040	0b00110000000001000000	0x30	0x1	Compulsory Miss	
d[8]	0x40040	0b01000000000001000000	0x40	0x1	Compulsory Miss	
e[8]	0x50040	0b01010000000001000000	0x50	0x1	Compulsory Miss	
a[9]	0x10048	0b00010000000001001000	0x10	0x1	Hit	
b[9]	0x20048	0b00100000000001001000	0x20	0x1	Hit	
c[9]	0x30048	0b00110000000001001000	0x30	0x1	Hit	
d[9]	0x40048	0b01000000000001001000	0x40	0x1	Hit	

intel Core i7

- D-L1 Cache configuration of intel Core i7
 - Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the major type of cache misses when running the above code snippet?

- A. Compulsory misses
- B. Conflict misses
- C. Capacity misses
- D. Capacity and conflict misses
- E. They are all equally critical

**How can programmer improve
memory performance?**

Data structures

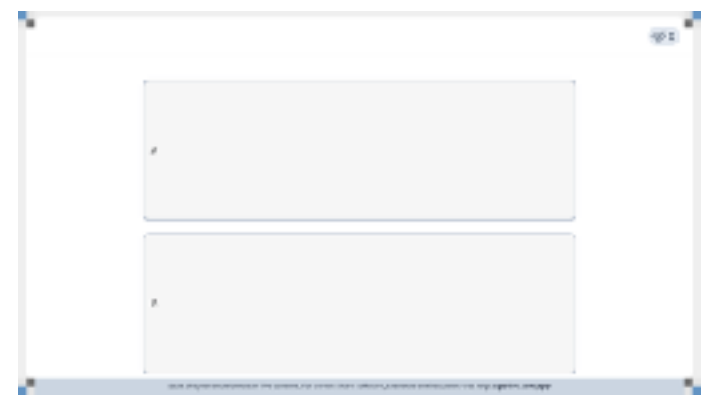


Column-store or row-store

- Considering your the most frequently used queries in your database system are similar to
`SELECT AVG(assignment_1) FROM table`
Which of the following would be a data structure that better implements the table supporting this type of queries?

Array of objects	object of arrays
<pre>struct grades { int id; double *homework; double average; }; table = (struct grades *) \ malloc(num_of_students*sizeof(struct</pre>	<pre>struct grades { int *id; double **homework; double *average; }; table =(struct grades *)malloc(sizeof(struct grades));</pre>

- A. Array of objects
- B. Object of arrays



Column-store or row-store

- Considering your the most frequently used queries in your database system are similar to

`SELECT AVG(assignment_1) FROM table`

Which of the following would be a data structure that better implements the table supporting this type of queries?

Array of objects	object of arrays
<pre>struct grades { int id; double *homework; double average; }; table = (struct grades *) \ malloc(num_of_students*sizeof(struct</pre>	<pre>struct grades { int *id; double **homework; double *average; }; table =(struct grades *)malloc(sizeof(struct grades));</pre>

A. Array of objects **What if we want to calculate average scores for each student?**

B. Object of arrays

Array of structures or structure of arrays

Array of objects			object of arrays												
	<pre>struct grades { int id; double *homework; double average; };</pre>			<pre>struct grades { int *id; double **homework; double *average; };</pre>											
	ID	*homework	average	ID	*homework	average									
average of each homework	<pre>for(i=0;i<homework_items; i++) { gradesheet[total_number_students].homework[i] = 0.0; for(j=0;j<total_number_students;j++) gradesheet[total_number_students].homework[i] +=gradesheet[j].homework[i]; gradesheet[total_number_students].homework[i] /= (double)total_number_students; }</pre>			<table><tr><td>ID</td><td>ID</td><td>ID</td></tr><tr><td>homework</td><td>homework</td><td>homework</td></tr><tr><td>average</td><td>average</td><td>average</td></tr></table> <pre>for(i = 0;i < homework_items; i++) { gradesheet.homework[i][total_number_students] = 0.0; for(j = 0; j <total_number_students;j++) { gradesheet.homework[i][total_number_students] += gradesheet.homework[i][j]; } gradesheet.homework[i][total_number_students] /= total_number_students; }</pre>			ID	ID	ID	homework	homework	homework	average	average	average
	ID	ID	ID												
homework	homework	homework													
average	average	average													

Column-store or row-store

- If you're designing an in-memory database system, will you be using

RowId	Empld	Lastname	Firstname	Salary
1	10	Smith	Joe	40000
2	12	Jones	Mary	50000
3	11	Johnson	Cathy	44000
4	22	Jones	Bob	55000

- column-store — stores data tables column by column

10:001, 12:002, 11:003, 22:004;
Smith:001, Jones:002, Johnson:003, Jones:004,
Joe:001, Mary:002, Cathy:003, Bob:004;
40000:001, 50000:002, 44000:003, 55000:004;

if the most frequently used query looks like –
select Lastname, Firstname from table

- row-store — stores data tables row by row

001:10, Smith, Joe, 40000;
002:12, Jones, Mary, 50000;
003:11, Johnson, Cathy, 44000;
004:22, Jones, Bob, 55000;

Takeaways: Software Optimizations

- Data layout — capacity miss, conflict miss, compulsory miss

Loop interchange/fission/fusion

Demo — programmer & performance

A

```
for(i = 0; i < ARRAY_SIZE; i++)
{
    for(j = 0; j < ARRAY_SIZE; j++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)
{
    for(i = 0; i < ARRAY_SIZE; i++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

$O(n^2)$

Complexity

$O(n^2)$

Same

Instruction Count?

Same

Same

Clock Rate

Same

Better

CPI

Worse

Loop optimizations

Loop interchange

A

```
for(i = 0; i < ARRAY_SIZE; i++)  
{  
    for(j = 0; j < ARRAY_SIZE; j++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```



B

```
for(j = 0; j < ARRAY_SIZE; j++)  
{  
    for(i = 0; i < ARRAY_SIZE; i++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

Takeaways: Software Optimizations

- Data layout — capacity miss, conflict miss, compulsory miss
- Loop interchange — conflict/capacity miss

NVIDIA Tegra Orin

- D-L1 Cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384], d[16384], e[16384];
/* a = 0x10000, b = 0x20000, c = 0x30000, d = 0x40000, e = 0x50000 */
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
    //load a[i], b[i], c[i], d[i] and then store to e[i]
}
```

What's the data cache miss rate for this code?

- A. 12.5%
- B. 56.25%
- C. 66.67%
- D. 68.75%
- E. 100%



What if the code look like this?

- D-L1 Cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384];
/* c = 0x10000, a = 0x20000, b = 0x30000 */
for(i = 0; i < 512; i++)
    e[i] = a[i] * b[i] + c[i]; //load a, b, c and then store to e
for(i = 0; i < 512; i++)
    e[i] /= d[i]; //load e, load d, and then store to e
```

What's the data cache miss rate for this code?

- A. ~10%
- B. ~20%
- C. ~40%
- D. ~80%
- E. 100%



What if the code look like this?

- D-L1 Cache configuration of NVIDIA Tegra Orin
 - Size 64KB, 4-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384], b[16384], c[16384];  
/* c = 0x10000, a = 0x20000, b = 0x30000 */  
for(i = 0; i < 512; i++)  
    e[i] = a[i] * b[i] + c[i]; //load a, b, c and then store to e  
for(i = 0; i < 512; i++)  
    e[i] /= d[i]; //load e, load d, and then store to e
```

What's the data cache miss rate for this code?

A. ~10%

B. ~20%

C. ~40%

D. ~80%

E. 100%

Loop optimizations

Loop interchange

A

```
for(i = 0; i < ARRAY_SIZE; i++)  
{  
    for(j = 0; j < ARRAY_SIZE; j++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)  
{  
    for(i = 0; i < ARRAY_SIZE; i++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```



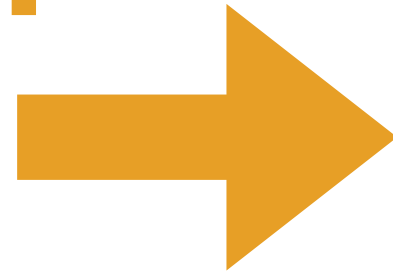
B

```
double a[8192], b[8192], c[8192], \  
       d[8192], e[8192];  
for(i = 0; i < 512; i++) {  
    e[i] = (a[i] * b[i] + c[i])/d[i];  
}
```

Loop fission

A

```
double a[8192], b[8192], c[8192], \  
       d[8192], e[8192];  
for(i = 0; i < 512; i++)  
    e[i] = a[i] * b[i] + c[i];  
for(i = 0; i < 512; i++)  
    e[i] /= d[i];
```



Takeaways: Software Optimizations

- Data layout — capacity miss, conflict miss, compulsory miss
- Loop interchange — conflict/capacity miss
- Loop fission — conflict miss — when \$ has limited way associativity



What if we change the processor?

- If we have an intel processor with a 48KB, 12-way, 64B-blocked L1 cache, which version of code performs better?
 - A. Version A, because the code incurs fewer cache misses
 - B. Version B, because the code incurs fewer cache misses
 - C. Version A, because the code incurs fewer memory references
 - D. Version B, because the code incurs fewer memory references
 - E. They are about the same

A

```
double a[8192], b[8192], c[8192], \
       d[8192], e[8192];
for(i = 0; i < 512; i++)
    e[i] = a[i] * b[i] + c[i];
for(i = 0; i < 512; i++)
    e[i] /= d[i];
```

B

```
double a[8192], b[8192], c[8192], \
       d[8192], e[8192];
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
}
```

What if we change the processor?

- If we have an intel processor with a 32KB, 8-way, 64B-blocked L1 cache, which version of code performs better?
 - A. Version A, because the code incurs fewer cache misses
 - B. Version B, because the code incurs fewer cache misses
 - C. Version A, because the code incurs fewer memory references
 - D. Version B, because the code incurs fewer memory references**
 - E. They are about the same

A

```
double a[8192], b[8192], c[8192], \
      d[8192], e[8192];
for(i = 0; i < 512; i++)
    e[i] = a[i] * b[i] + c[i];
for(i = 0; i < 512; i++)
    e[i] /= d[i];
```

B

```
double a[8192], b[8192], c[8192], \
      d[8192], e[8192];
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
}
```

Loop optimizations

Loop interchange

A

```
for(i = 0; i < ARRAY_SIZE; i++)
{
    for(j = 0; j < ARRAY_SIZE; j++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)
{
    for(i = 0; i < ARRAY_SIZE; i++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```



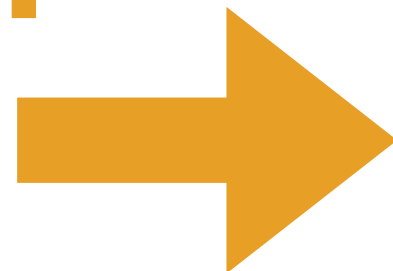
B

```
double a[8192], b[8192], c[8192], \
       d[8192], e[8192];
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
}
```

Loop fission

A

```
double a[8192], b[8192], c[8192], \
       d[8192], e[8192];
for(i = 0; i < 512; i++)
    e[i] = a[i] * b[i] + c[i];
for(i = 0; i < 512; i++)
    e[i] /= d[i];
```



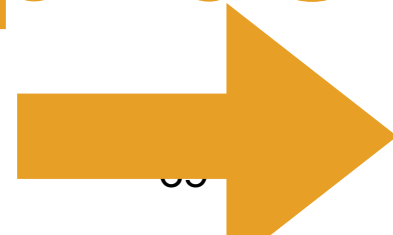
A

```
double a[8192], b[8192], c[8192], \
       d[8192], e[8192];
for(i = 0; i < 512; i++)
    e[i] = a[i] * b[i] + c[i];
for(i = 0; i < 512; i++)
    e[i] /= d[i];
```

Loop fusion

B

```
double a[8192], b[8192], c[8192], \
       d[8192], e[8192];
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
}
```



Takeaways: Software Optimizations

- Data layout — capacity miss, conflict miss, compulsory miss
- Loop interchange — conflict/capacity miss
- Loop fission — conflict miss — when \$ has limited way associativity
- Loop fusion — capacity miss — when \$ has enough way associativity

Announcement

- Check your grades on https://www.escalab.org/my_grades
- Assignment #2 due **this Sunday**
 - You should run the performance measurement yourself and calculate results based on that — everyone should have a different answer
 - All questions this time require **correct** estimations in cache performance to help you better prepare the examines
- Midterm
 - Schedule with Triton Testing Center through <https://us.prairietest.com/pt/>
 - Testing center will handle the testing related thing. You are responsible for being there on time and following the rules. Contact Testing center if you need additional assistance. I don't handle those
 - Practice test will be really helpful for you to get used to the feeling of the midterm
 - 45 minutes, no cheatsheet or outside materials allowed
 - More details to come tomorrow
- Reading quiz #5 due **next Tuesday** before the lecture — all reading quizzes are on Gradescope

Computer Science & Engineering

142

つづく

